

ADK 與多模態工具互動：第 2 部分 (含工具回呼的 MCP 工具組)

來源：<https://codelabs.developers.google.com/adk-multimodal-tool-part-2?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何處理使用者上傳資料、工具輸入內容和 MCP Toolset 工具回應之間的多模態資料流。

目錄

1.  簡介
2.  (Optional) Preparing Workshop Development Setup
3.  初始化 Veo MCP 伺服器
4.  將 Veo MCP 伺服器連線至 ADK 代理
5.  修改工具呼叫參數
6.  修改工具回應
7.  摘要
8.  清理

步驟 1 · 約 0 分鐘

1. 簡介

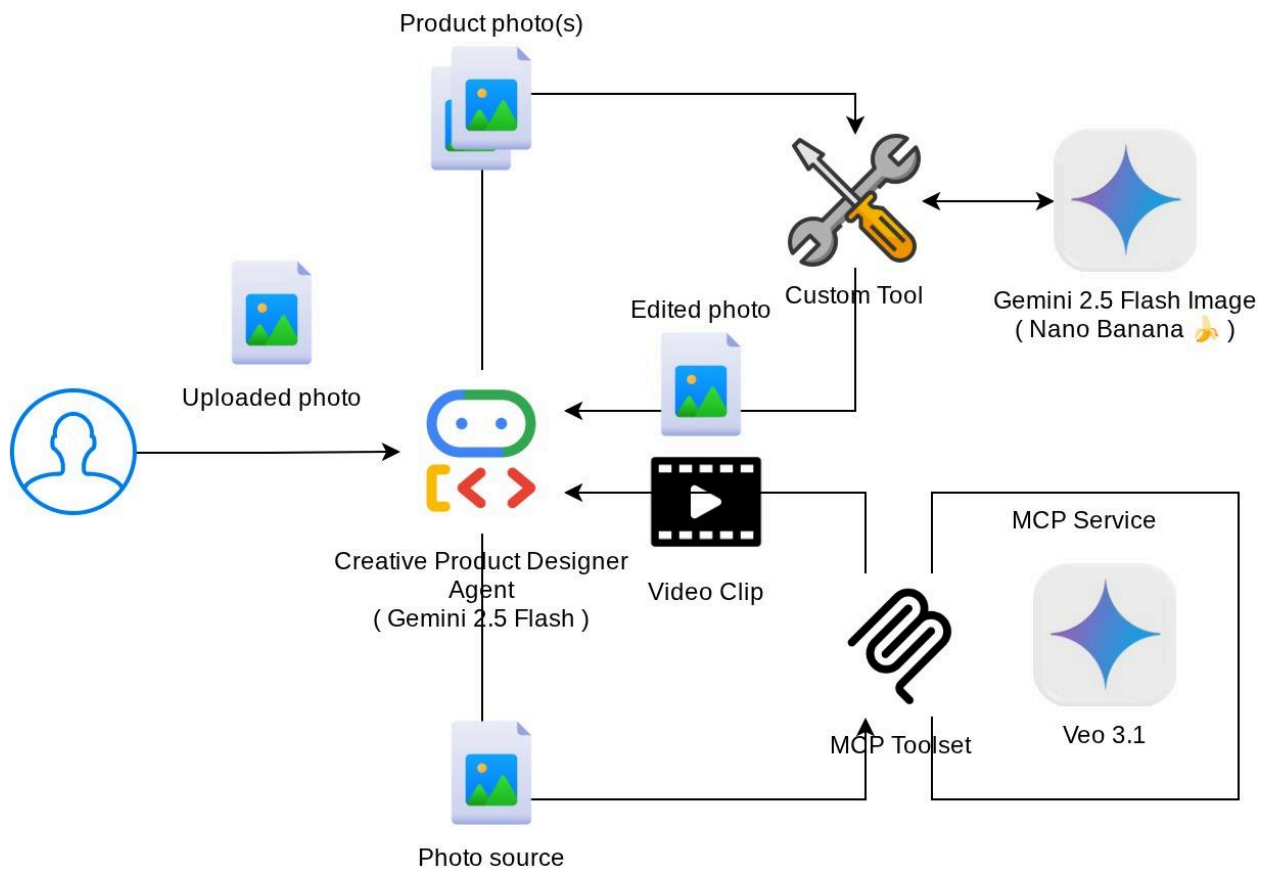
在先前的[程式碼研究室](#)中，您已瞭解如何在 ADK 中設計多模態資料互動。現在，我們將進一步瞭解如何使用 MCP 工具組，設計與 MCP 伺服器之多模態資料互動。我們將擴充先前開發的產品相片編輯器代理程式功能，透過 Veo MCP 伺服器使用 Veo 模型生成短片

在本程式碼研究室中，您將逐步完成下列步驟：

1. 準備 Google Cloud 雲端專案和基礎代理程式目錄
2. 設定需要檔案資料做為輸入內容的 MCP 伺服器
3. 讓 ADK 代理連線至 MCP 伺服器
4. 設計提示策略和回呼函式，修改對 MCP Toolset 的函式呼叫要求
5. 設計回呼函式，處理 MCP 工具組之多模態資料回應

架構總覽

本程式碼研究室的整體互動方式如下圖所示



必要條件

- 熟悉 Python
- (選用) Agent Development Kit (ADK) 基礎程式碼研究室
 1. goo.gle/adk-foundation
 2. goo.gle/adk-using-tools
- (選用) ADK 多模態工具第 1 部分程式碼研究室：goo.gle/adk-multimodal-tool-1

課程內容

- 如何使用提示詞和圖片，透過 Veo 3.1 製作短片
- 如何使用 FastMCP 開發多模態 MCP 伺服器
- 如何設定 ADK 來使用 MCP Toolset
- 如何透過工具回呼，將工具呼叫修改為 MCP 工具組
- 如何透過工具回呼，修改 MCP 工具組的工具回應

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用帳單帳戶的 Cloud 專案

本程式碼研究室適合各種程度的開發人員 (包括初學者)，並使用 Python 撰寫範例應用程式。不過，您不需要具備 Python 知識，也能瞭解本文介紹的概念。

2. 🚀 (Optional) Preparing Workshop Development Setup

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

重要附註：如果您要繼續進行第 1 部分程式碼實驗室的教學課程，可以略過本節。

目標：確認專案和帳單帳戶已準備就緒、熟悉 Cloud Shell 終端機和編輯器，並設定工作目錄和基本代理程式

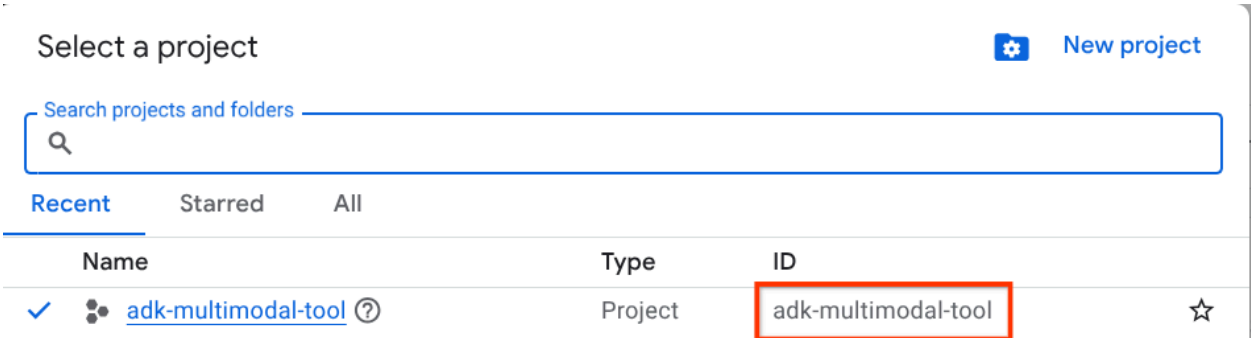
重要注意事項：
本教學課程需要具備有效帳單帳戶的 Google Cloud 專案。如果沒有，請使用本程式碼研究室頂端的橫幅，取得可用於本教學課程的試用帳單帳戶。

步驟 1：在 Cloud 控制台中選取有效專案

在 Google Cloud 控制台的專案選取器頁面中，選取或建立 Google Cloud 專案 (請參閱控制台左上方的部分)



點選該按鈕後，您會看到所有專案的清單，如下例所示：



紅框標示的值是專案 ID，本教學課程會使用這個值。

確認 Cloud 專案已啟用計費功能。如要確認，請按一下左上列的漢堡圖示 ≡，顯示「導覽選單」，然後找出「帳單」選單



Billing



如果「帳單 / 總覽」標題下方 (**Cloud 控制台左上角部分**) 顯示「Google Cloud Platform 試用帳單帳戶」，表示您的專案已準備就緒，可供本教學課程使用。如果沒有，請返回本教學課程的開頭，兌換試用帳單帳戶



Billing / Overview

Billing account

Google Cloud Platform Trial Billing

步驟 2：熟悉 Cloud Shell

您將在教學課程的大部分環節中使用 [Cloud Shell](#)，請點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。如果系統提示您授權，請點選「授權」。

Search (/) for resources, docs, products, and ...

Search



Click on the "Cloud Shell" button



Build with
Gemini Co

Ask Gemini in Cl
Editor.

Authorize Cloud Shell

Cloud Shell needs permission to use your credentials to make Google Cloud API calls.

Click Authorize to grant permission to this and future calls.

Reject

Authorize

連至 Cloud Shell 後，我們需要檢查 Shell (或終端機) 是否已通過帳戶驗證

```
gcloud auth list
```

如果看到如下列範例輸出內容的個人 Gmail，表示一切正常

Credentialed Accounts

ACTIVE: *

ACCOUNT: alvinprayuda@gmail.com

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

如果沒有，請嘗試重新整理瀏覽器，並確保在系統提示時點選「授權」（連線問題可能會導致授權中斷）。

接著，我們也需要檢查 Shell 是否已設定為正確的專案 ID。如果終端機的 \$ 圖示前有括號內的值 (在下方螢幕截圖中，該值為「adk-multimodal-tool」)，表示目前 Shell 工作階段已設定專案。

```
alvinprayuda@cloudshell:~ (adk-multimodal-tool)$
```

如果顯示的值正確無誤，可以略過下一個指令。但如果該值不正確或遺失，請執行下列指令

```
gcloud config set project <YOUR_PROJECT_ID>
```

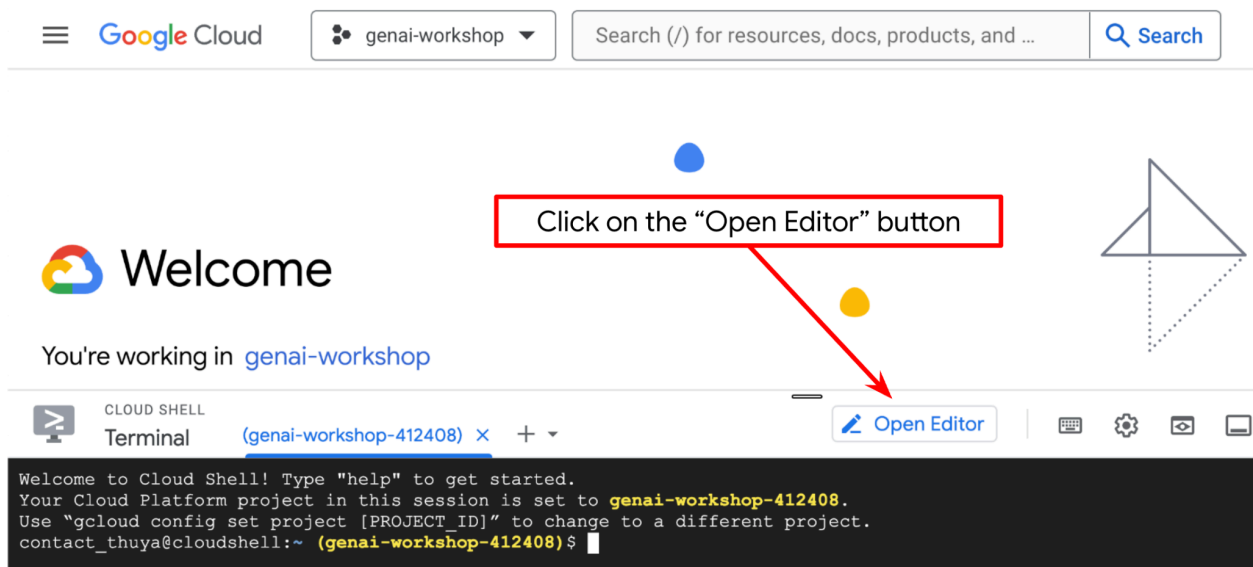
接著，從 Github 複製本程式碼研究室的範本工作目錄，執行下列指令。系統會在 **adk-multimodal-tool** 目錄中建立工作目錄

```
git clone https://github.com/alphinside/adk-mcp-multimodal.git adk-multimodal-tool
```

步驟 3：熟悉 Cloud Shell 編輯器並設定應用程式工作目錄

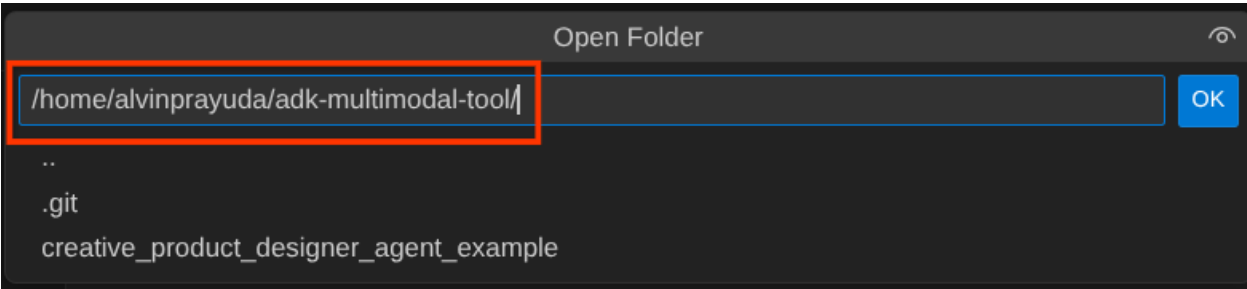
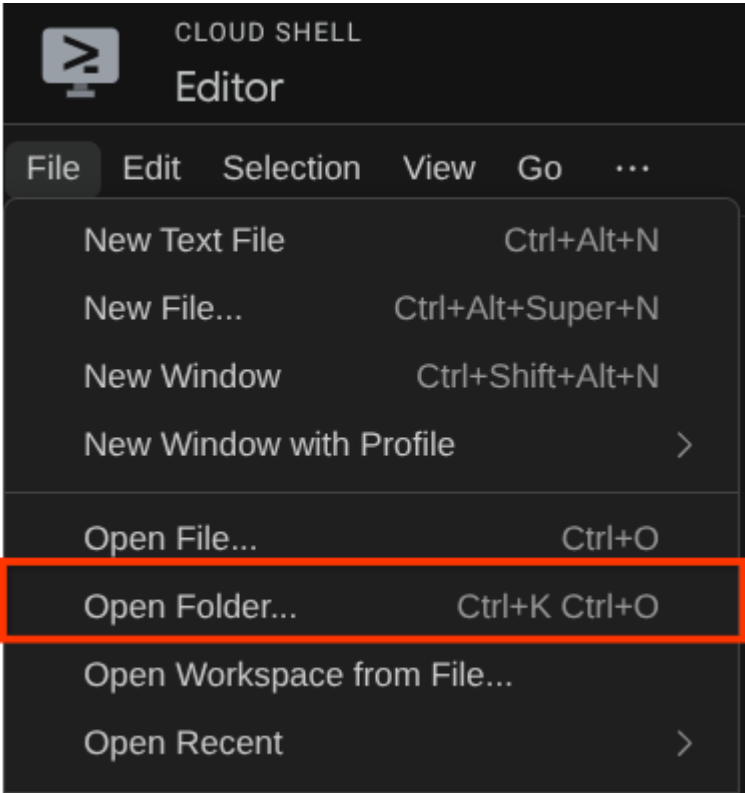
現在，我們可以設定程式碼編輯器，進行一些程式設計工作。我們會使用 Cloud Shell 編輯器執行這項操作

按一下「Open Editor」（開啟編輯器）按鈕，開啟 Cloud Shell 編輯器

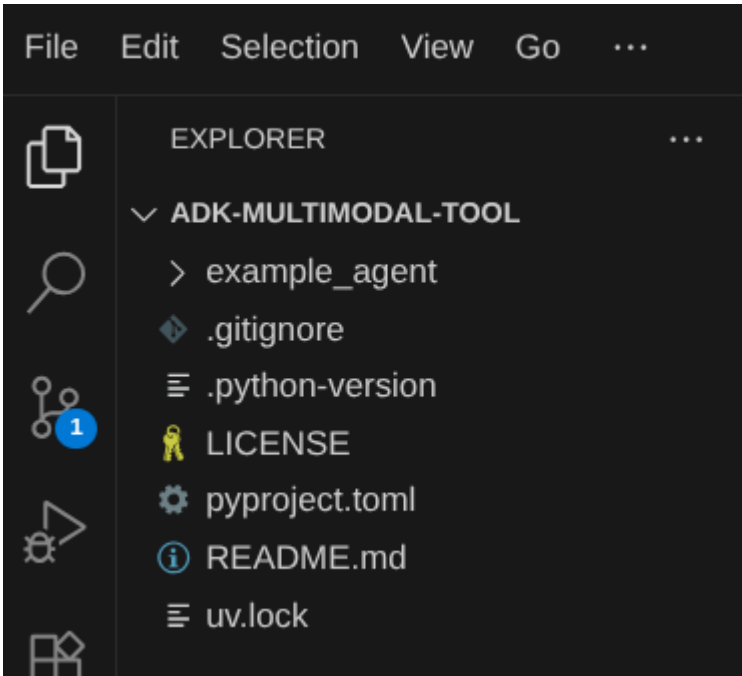


完成後，前往 Cloud Shell 編輯器頂端，依序點選「File」->「Open Folder」，找出「username」目錄，然後找出「adk-multimodal-tool」目錄，並點選「OK」按鈕。這會將所選目錄設為主要工作目錄。在本範例中，使用者名稱為

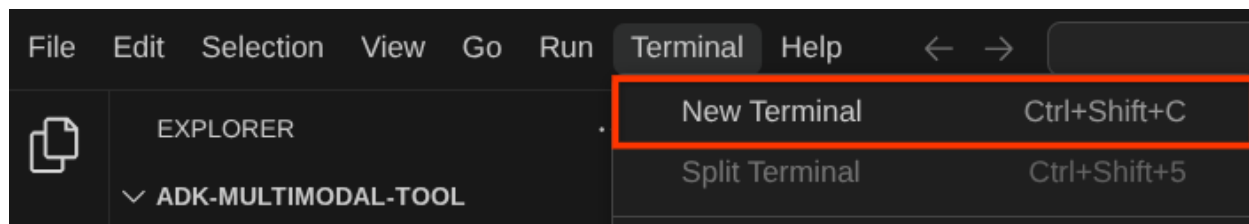
alvinprayuda，因此目錄路徑如下所示



現在，Cloud Shell 編輯器的工作目錄應如下所示 (位於 **adk-multimodal-tool** 內)：



現在，請開啟編輯器的終端機。方法是在選單列中依序點選「Terminal」->「New Terminal」，或使用 **Ctrl + Shift + C** 鍵盤快速鍵，瀏覽器底部就會開啟終端機視窗。



目前啟用的終端機應位於 **adk-multimodal-tool** 工作目錄中。在本程式碼研究室中，我們將使用 Python 3.12，並使用 [uv Python 專案管理工具](#)，簡化建立及管理 Python 版本和虛擬環境的需求。Cloud Shell 已預先安裝 **uv** 套件。

注意：使用 **uv** 做為 Python 專案管理員時，建立及啟用**虛擬環境**的程序會簡化。不過，每個 **python** 指令碼指令都會替換為 **uv run** 指令。例如：**uv run main.py**，而不是 **python main.py**

執行此指令，將必要的依附元件安裝至 **.venv** 目錄的虛擬環境

```
uv sync --frozen
```

查看 **pyproject.toml**，瞭解本教學課程中宣告的依附元件 (即 `google-adk`, and `python-dotenv`)。

注意：Python 依附元件會在 **pyproject.toml** 中宣告，並在 **uv.lock** 檔案中鎖定版本

現在，我們需要透過下列指令啟用必要的 API。這可能需要一些時間。

```
gcloud services enable aiplatform.googleapis.com
```

成功執行指令後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```

複製的存放區中，**part2_starter_agent** 目錄內已提供範本代理程式結構。現在，我們需要先重新命名，才能開始本教學課程

```
mv part1_ckpt_agent product_photo_editor
```

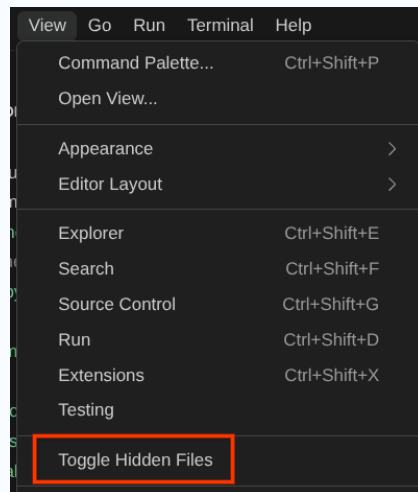
接著，將 **product_photo_editor/.env.example** 複製到 **product_photo_editor/.env**

```
cp product_photo_editor/.env.example product_photo_editor/.env
```

開啟 **product_photo_editor/.env** 檔案後，您會看到如下所示的內容

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=global
```

注意：如果編輯器未顯示 `.env` 檔案，表示該檔案屬於隱藏的檔案。依序點選「View」->「Toggle Hidden Files」即可顯示



然後，您需要使用正確的專案 ID 更新 `your-project-id` 值。現在可以進行下一個步驟了

步驟 3 · 約 0 分鐘

3. 🚀 初始化 Veo MCP 伺服器

目標：在本節中，您將使用 FastMCP 開發 MCP 伺服器，將提示傳送至 Veo 模型來製作影片

注意：請確認目前的工作目錄位於 **adk-multimodal-tool** 工作目錄中

首先，請使用下列指令建立 MCP 服務目錄

```
mkdir veo_mcp
```

然後使用這個指令建立 **veo_mcp/main.py**

```
touch veo_mcp/main.py
```

然後將下列程式碼複製到 **veo_mcp/main.py**

```

from fastmcp import FastMCP
from typing import Annotated
from pydantic import Field
import base64
import asyncio
import os
from google import genai
from google.genai import types
from dotenv import load_dotenv
import logging

# Load environment variables from .env file
load_dotenv()

mcp = FastMCP("Veo MCP Server")

@mcp.tool
async def generate_video_with_image(
    prompt: Annotated[
        str, Field(description="Text description of the video to generate")
    ],
    image_data: Annotated[
        str, Field(description="Base64-encoded image data to use as starting frame")
    ],
    negative_prompt: Annotated[
        str | None,
        Field(description="Things to avoid in the generated video"),
    ] = None,
) -> dict:
    """Generates a professional product marketing video from text prompt and starting image
    using Google's Veo API.

```

This function uses an image as the first frame of the generated video and automatically enriches your prompt with professional video production quality guidelines to create high-quality marketing assets suitable for commercial use.

AUTOMATIC ENHANCEMENTS APPLIED:

- 4K cinematic quality with professional color grading
- Smooth, stabilized camera movements
- Professional studio lighting setup
- Shallow depth of field for product focus
- Commercial-grade production quality
- Marketing-focused visual style

PROMPT WRITING TIPS:

Describe what you want to see in the video. Focus on:

- Product actions/movements (e.g., "rotating slowly", "zooming into details")
- Desired camera angles (e.g., "close-up of the product", "wide shot")
- Background/environment (e.g., "minimalist white backdrop", "lifestyle setting")
- Any specific details about the product presentation

The system will automatically enhance your prompt with professional production quality.

Args:

prompt: Description of the video to generate. Focus on the core product presentation you want. The system will automatically add professional quality enhancements.

image_data: Base64-encoded image data to use as the starting frame

negative_prompt: Optional prompt describing what to avoid in the video

Returns:

dict: A dictionary containing:

- **status:** 'success' or 'error'
- **message:** Description of the result
- **video_data:** Base64-encoded video data (on success only)

"""

try:

```
# Initialize the Gemini client
client = genai.Client(
    vertexai=True,
    project=os.getenv("GOOGLE_CLOUD_PROJECT"),
    location=os.getenv("GOOGLE_CLOUD_LOCATION"),
)

# Decode the image
image_bytes = base64.b64decode(image_data)
print(f"Successfully decoded image data: {len(image_bytes)} bytes")

# Create image object
image = types.Image(image_bytes=image_bytes, mime_type="image/png")

# Prepare the config
config = types.GenerateVideosConfig(
    duration_seconds=8,
    number_of_videos=1,
)

if negative_prompt:
    config.negative_prompt = negative_prompt

# Enrich the prompt for professional marketing quality
enriched_prompt = enrich_prompt_for_marketing(prompt)

# Generate the video (async operation)
operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=enriched_prompt,
    image=image,
    config=config,
)

# Poll until the operation is complete
poll_count = 0
```

```

while not operation.done:
    poll_count += 1
    print(f"Waiting for video generation to complete... (poll {poll_count})")
    await asyncio.sleep(5)
    operation = client.operations.get(operation)

# Download the video and convert to base64
video = operation.response.generated_videos[0]

# Get video bytes and encode to base64
video_bytes = video.video.video_bytes
video_base64 = base64.b64encode(video_bytes).decode("utf-8")

print(f"Video generated successfully: {len(video_bytes)} bytes")

return {
    "status": "success",
    "message": f"Video with image generated successfully after {poll_count * 5}
seconds",
    "complete_prompt": enriched_prompt,
    "video_data": video_base64,
}
except Exception as e:
    logging.error(e)
    return {
        "status": "error",
        "message": f"Error generating video with image: {str(e)}",
    }

def enrich_prompt_for_marketing(user_prompt: str) -> str:
    """Enriches user prompt with professional video production quality enhancements.

    Adds cinematic quality, professional lighting, smooth camera work, and marketing-focused
    elements to ensure high-quality product marketing videos.
    """
    enhancement_prefix = """Create a high-quality, professional product marketing video with
the following characteristics:

TECHNICAL SPECIFICATIONS:
- 4K cinematic quality with professional color grading
- Smooth, stabilized camera movements
- Professional studio lighting setup with soft, even illumination
- Shallow depth of field for product focus
- High dynamic range (HDR) for vibrant colors

VISUAL STYLE:
- Clean, minimalist aesthetic suitable for premium brand marketing
- Elegant and sophisticated presentation
- Commercial-grade production quality
- Attention to detail in product showcase

```

```
USER'S SPECIFIC REQUIREMENTS:
```

```
"""
```

```
    enhancement_suffix = ""
```

```
ADDITIONAL QUALITY GUIDELINES:
```

- Ensure smooth transitions and natural motion
- Maintain consistent lighting throughout
- Keep the product as the clear focal point
- Use professional camera techniques (slow pans, tracking shots, or dolly movements)
- Apply subtle motion blur for cinematic feel
- Ensure brand-appropriate tone and style"""

```
    return f"{enhancement_prefix}{user_prompt}{enhancement_suffix}"
```

```
if __name__ == "__main__":
```

```
    mcp.run()
```

下列程式碼會執行下列動作：

1. 建立 FastMCP 伺服器，向 ADK 代理公開 Veo 3.1 影片生成工具
2. 接受以 base64 編碼的圖片、文字提示和負面提示做為輸入內容
3. 透過向 Veo 3.1 API 提交要求，並每 5 秒輪詢一次，直到完成為止，以非同步方式生成 8 秒影片
4. 傳回 base64 編碼的影片資料和充實過的提示

這個 Veo MCP 工具會與代理程式使用相同的環境變數，因此我們只要複製貼上 `.env` 檔案即可。執行下列指令：

```
cp product_photo_editor/.env veo_mcp/
```

現在，我們可以執行這項指令，測試 MCP 伺服器是否正常運作

```
uv run veo_mcp/main.py
```

並顯示類似這樣的控制台記錄

4. 🚀 將 Veo MCP 伺服器連線至 ADK 代理

目標：初始化代理使用的 MCP 工具集

現在，讓我們連線 Veo MCP 伺服器，以便代理使用。首先，請建立另一個指令碼來包含工具集，執行下列指令：

```
touch product_photo_editor/mcp_tools.py
```

然後將下列程式碼複製到 **product_photo_editor/mcp_tools.py**

```
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

mcp_toolset = MCPToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command="uv",
            args=[
                "run",
                "veo_mcp/main.py",
            ],
        ),
        timeout=120, # seconds
    ),
)

# Option to connect to remote MCP server

# from google.adk.tools.mcp_tool.mcp_session_manager import StreamableHTTPConnectionParams

# mcp_toolset = MCPToolset(
#     connection_params=StreamableHTTPConnectionParams(
#         url="http://localhost:8000/mcp",
#         timeout=120,
#     ),
# )
```

上述程式碼顯示如何使用 ADK MCPToolset 連線至 MCP 伺服器。在本範例中，我們使用 STDIO 通訊管道連線至 MCP 伺服器。我們會在指令中指定如何執行 MCP 伺服器，並設定逾時參數。

注意：您也可以使用 `StreamableHTTPConnection` 連線至遠端 MCP 伺服器，如註解程式碼範例所示。詳情請參閱 [這份說明文件](#)。

5. 🚀 修改工具呼叫參數

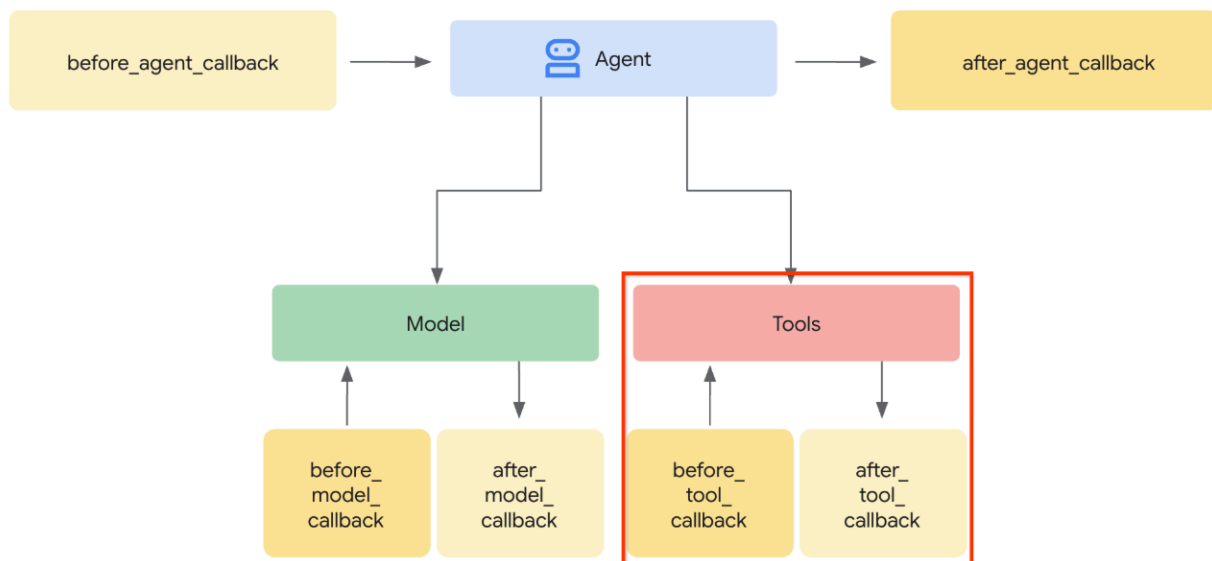
目標：處理 MCP 工具的 base64 引數輸入內容

在 MCP 伺服器工具宣告中，我們設計了工具 `generate_video_with_image`，將 base64 字串指定為工具參數。我們無法要求 LLM 為我們執行這項操作，因此需要設計特定策略來處理。

在上一個實驗室中，我們會在 `before_model_callback` 中處理使用者上傳的圖片和工具回應圖片，並將其儲存為構件，這也會反映在先前準備的代理程式範本中。我們會運用這項資訊，採取下列策略：

1. 如果特定工具參數需要傳送 Base64 字串資料，請指示 LLM 一律傳送 `artifact_id` 值
2. 在 `before_tool_callback` 中攔截工具呼叫的叫用，並載入構件，將參數從 `artifact_id` 轉換為位元組內容，然後覆寫工具引數

請參閱下圖，瞭解我們要攔截的部分



首先，請準備 `before_tool_callback` 函式，然後執行下列指令，建立新的 `product_photo_editor/tool_callbacks.py` 檔案

```
touch product_photo_editor/tool_callbacks.py
```

然後將下列程式碼複製到檔案中

```
# product_photo_editor/tool_callbacks.py

from google.genai.types import Part
from typing import Any
from google.adk.tools.tool_context import ToolContext
from google.adk.tools.base_tool import BaseTool
from google.adk.tools.mcp_tool.mcp_tool import McpTool
import base64
import logging
import json
from mcp.types import CallToolResult

async def before_tool_modifier(
    tool: BaseTool, args: dict[str, Any], tool_context: ToolContext
):
    # Identify which tool input should be modified
    if isinstance(tool, McpTool) and tool.name == "generate_video_with_image":
        logging.info("Modify tool args for artifact: %s", args["image_data"])
        # Get the artifact filename from the tool input argument
        artifact_filename = args["image_data"]
        artifact = await tool_context.load_artifact(filename=artifact_filename)
        file_data = artifact.inline_data.data

        # Convert byte data to base64 string
        base64_data = base64.b64encode(file_data).decode("utf-8")

        # Then modify the tool input argument
        args["image_data"] = base64_data
```

上述程式碼顯示下列步驟：

1. 檢查叫用的工具是否為 `McpTool` 物件，以及是否為要修改的目標工具呼叫
2. 以 Base64 格式取得 `image_data` 引數的值，但我們要求 LLM 傳回其中的 `artifact_id`
3. 在 `tool_context` 上使用構件服務載入構件
4. 使用 base64 資料覆寫 `image_data` 引數

現在，我們需要將這個回呼新增至代理程式，並稍微修改指令，讓代理程式一律以構件 ID 填入 Base64 工具引數。

開啟 `product_photo_editor/agent.py`，並使用下列程式碼修改內容

```
# product_photo_editor/agent.py

from google.adk.agents.llm_agent import Agent
from product_photo_editor.custom_tools import edit_product_asset
from product_photo_editor.mcp_tools import mcp_toolset
from product_photo_editor.model_callbacks import before_model_modifier
from product_photo_editor.tool_callbacks import before_tool_modifier
from product_photo_editor.prompt import AGENT_INSTRUCTION

root_agent = Agent(
    model="gemini-2.5-flash",
    name="product_photo_editor",
    description="""A friendly product photo editor assistant that helps small business
owners edit and enhance their product photos. Perfect for improving photos of handmade
goods, food products, crafts, and small retail items""",
    instruction=AGENT_INSTRUCTION
    + """
**IMPORTANT: Base64 Argument Rule on Tool Call**

If you found any tool call arguments that requires base64 data,
ALWAYS provide the artifact_id of the referenced file to
the tool call. NEVER ask user to provide base64 data.
Base64 data encoding process is out of your
responsibility and will be handled in another part of the system.
""",
    tools=[
        edit_product_asset,
        mcp_toolset,
    ],
    before_model_callback=before_model_modifier,
    before_tool_callback=before_tool_modifier,
)

```

現在，我們來試著與代理互動，測試這項修改。執行下列指令，即可執行網頁開發使用者介面

```
uv run adk web --port 8080
```

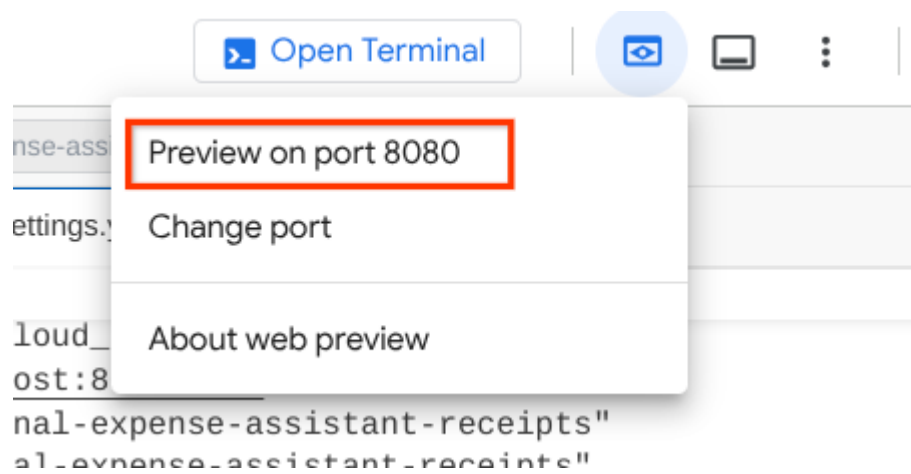
這會產生類似下列範例的輸出內容，表示我們已可存取網頁介面

```
INFO:      Started server process [xxxx]
INFO:      Waiting for application startup.

+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://127.0.0.1:8080. |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
```

現在，如要檢查，請按住 **Ctrl** 鍵並點選網址，或點選 Cloud Shell 編輯器頂端區域的「Web Preview」(網頁預覽) 按鈕，然後選取「Preview on port 8080」(透過以下通訊埠預覽：8080)。



你會看到以下網頁，在左上方的下拉式按鈕中選取可用的代理程式 (在本例中應為 **product_photo_editor**)，並與機器人互動。

接著上傳下列圖片，並要求代理程式從中生成宣傳短片

```
Generate a slow zoom in and moving from left and right animation
```



您會遇到下列錯誤

```
{"error": "400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': 'Unable to submit request because the input token count is 5596803 but model only supports up to 1048576. Reduce the input token count and try again. You can also use the CountTokens API to calculate prompt token count and billable characters. Learn more: https://cloud.google.com/vertex-ai/generative-ai/docs/learn/models', 'status': 'INVALID_ARGUMENT'}}"}
```

OK

ERROR is expected here

這是因為由於工具直接以 base64 字串的形式傳回結果，因此會超過權杖上限。現在，我們將在下一節中處理這項錯誤。

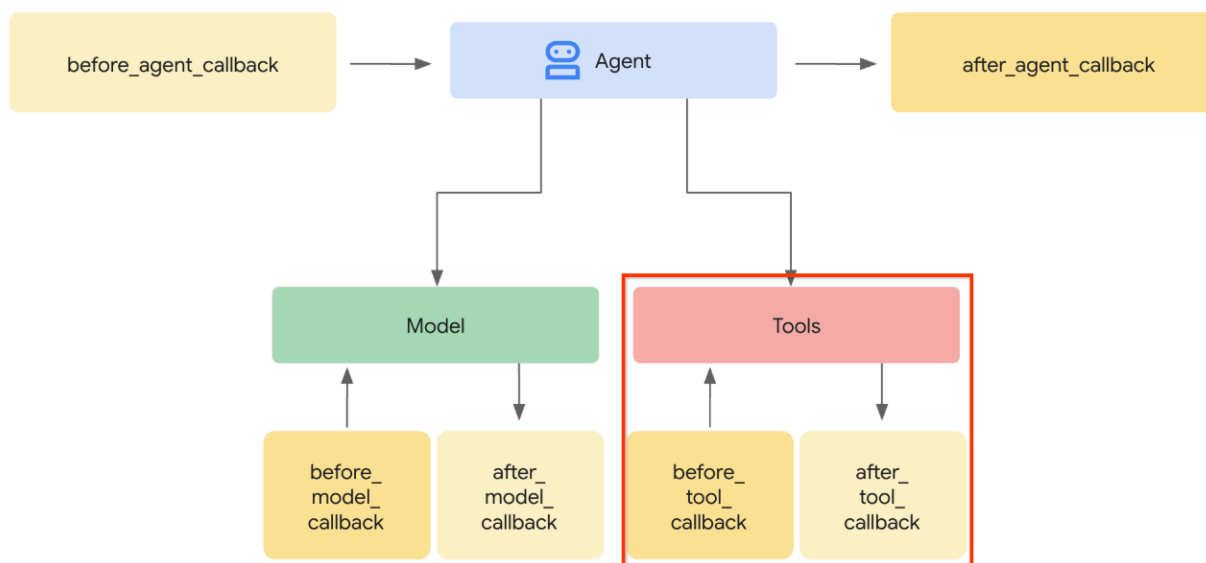
6. 🚀 修改工具回應

目標：處理 MCP 工具的 base64 資料回應

在本節中，我們將處理 MCP 回應中的工具回應。我們會執行下列動作：

1. 將工具產生的影片回應儲存在構件服務中
2. 改為將構件 ID 傳回給代理程式

提醒你，我們將在下列代理程式執行階段中輕觸



首先，實作回呼函式，開啟 `product_photo_editor/tool_callbacks.py` 並修改，以實作 `after_tool_modifier`


```

# product_photo_editor/tool_callbacks.py

from google.genai.types import Part
from typing import Any
from google.adk.tools.tool_context import ToolContext
from google.adk.tools.base_tool import BaseTool
from google.adk.tools.mcp_tool.mcp_tool import McpTool
import base64
import logging
import json
from mcp.types import CallToolResult

async def before_tool_modifier(
    tool: BaseTool, args: dict[str, Any], tool_context: ToolContext
):
    # Identify which tool input should be modified
    if isinstance(tool, McpTool) and tool.name == "generate_video_with_image":
        logging.info("Modify tool args for artifact: %s", args["image_data"])
        # Get the artifact filename from the tool input argument
        artifact_filename = args["image_data"]
        artifact = await tool_context.load_artifact(filename=artifact_filename)
        file_data = artifact.inline_data.data

        # Convert byte data to base64 string
        base64_data = base64.b64encode(file_data).decode("utf-8")

        # Then modify the tool input argument
        args["image_data"] = base64_data

async def after_tool_modifier(
    tool: BaseTool,
    args: dict[str, Any],
    tool_context: ToolContext,
    tool_response: dict | CallToolResult,
):
    if isinstance(tool, McpTool) and tool.name == "generate_video_with_image":
        tool_result = json.loads(tool_response.content[0].text)

        # Get the expected response field which contains the video data
        video_data = tool_result["video_data"]
        artifact_filename = f"video_{tool_context.function_call_id}.mp4"

        # Convert base64 string to byte data
        video_bytes = base64.b64decode(video_data)

        # Save the video as artifact
        await tool_context.save_artifact(
            filename=artifact_filename,
            artifact=Part(inline_data={"mime_type": "video/mp4", "data": video_bytes}),
        )

```

```

        # Remove the video data from the tool response
        tool_result.pop("video_data")

        # Then modify the tool response to include the artifact filename and remove the
base64 string
        tool_result["video_artifact_id"] = artifact_filename
        logging.info(
            "Modify tool response for artifact: %s", tool_result["video_artifact_id"]
        )

    return tool_result

```

接著，我們需要為代理程式配備這項函式。開啟 **product_photo_editor/agent.py**，並修改為下列程式碼

```

# product_photo_editor/agent.py

from google.adk.agents.llm_agent import Agent
from product_photo_editor.custom_tools import edit_product_asset
from product_photo_editor.mcp_tools import mcp_toolset
from product_photo_editor.model_callbacks import before_model_modifier
from product_photo_editor.tool_callbacks import (
    before_tool_modifier,
    after_tool_modifier,
)
from product_photo_editor.prompt import AGENT_INSTRUCTION

root_agent = Agent(
    model="gemini-2.5-flash",
    name="product_photo_editor",
    description="""A friendly product photo editor assistant that helps small business
owners edit and enhance their product photos. Perfect for improving photos of handmade
goods, food products, crafts, and small retail items""",
    instruction=AGENT_INSTRUCTION
    + """
**IMPORTANT: Base64 Argument Rule on Tool Call**

If you found any tool call arguments that requires base64 data,
ALWAYS provide the artifact_id of the referenced file to
the tool call. NEVER ask user to provide base64 data.
Base64 data encoding process is out of your
responsibility and will be handled in another part of the system.
""",
    tools=[
        edit_product_asset,
        mcp_toolset,
    ],
    before_model_callback=before_model_modifier,
    before_tool_callback=before_tool_modifier,
    after_tool_callback=after_tool_modifier,
)

```

完成後，你就能要求智慧助理編輯相片，甚至生成影片！再次執行下列指令

```
uv run adk web --port 8080
```

然後嘗試使用這張圖片製作影片

```
Generate a slow zoom in and moving from left and right animation
```



系統會生成影片 (如下方範例所示)，並儲存為構件

Agent Development Kit

product_photo_edit...

TraceEventsStateArtifactsSessionsEval

[usr_upl_img_196b00b51ac93840.png](#)

Version: 0Download



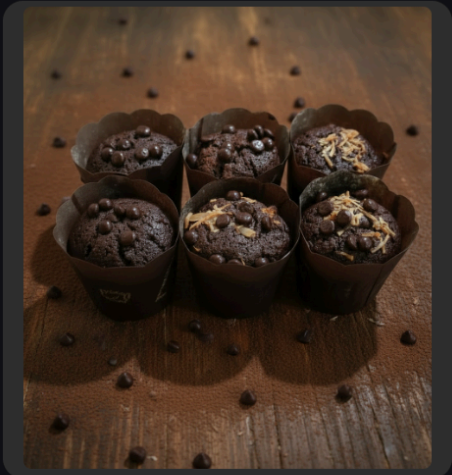
[video_adk-128e7c53-39e0-43b4-9b00-5c581d0f547f.mp4](#)

Version: 0Download

SESSION ID d46699a2-da42-4fcf-ba28-8196331bb0a8Token StreamingNew Session



DO



DO

⚡ generate_video_with_image

[video.mp4](#)

DO

✓ generate_video_with_image

DO

Great! I've generated a video for you. It features a slow, cinematic zoom into the chocolate muffins, with the camera gently panning from left to right, really highlighting their delicious textures and the scattered chocolate chips.

You can find your new video here: [video_adk-128e7c53-39e0-43b4-9b00-5c581d0f547f.mp4](#)

Let me know if you'd like any other adjustments or another video!

7. ★ 摘要

現在讓我們回顧一下在本程式碼研究室中完成的內容，以下是主要學習內容：

1. **多模態資料處理 (工具 I/O)**：強化策略，透過 ADK 的構件服務和專用回呼，管理工具輸入和輸出內容的多模態資料 (例如圖片和影片)，而非直接傳遞原始位元組資料。
 2. **整合 MCP 工具組**：透過 ADK MCP 工具組使用 FastMCP 開發及整合外部 Veo MCP 伺服器，為代理程式新增影片生成功能。
 3. **工具輸入內容修改 (before_tool_callback)**：實作回呼來攔截 generate_video_with_image 工具呼叫，將檔案的 artifact_id (由 LLM 選取) 轉換為 MCP 伺服器輸入內容所需的 base64 編碼圖片資料。
 4. **工具輸出內容修改 (after_tool_callback)**：實作回呼，攔截來自 MCP 伺服器的 Base64 編碼大型影片回應，將影片儲存為新構件，並將乾淨的 video_artifact_id 參照傳回 LLM。
-

步驟 8 · 約 0 分鐘

8. 🧹 清理

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-